

This is actually the **core of Company Brain**. If you understand this properly, you'll understand how your entire startup works.

I'll explain it like you're building **Google Search for a company**.

Imagine a Company

Suppose Microsoft connects Company Brain.

Their Google Drive has these folders:

Google Drive

HR/

Employee Handbook.pdf

Engineering/

Project Phoenix.docx

Finance/

Budget.xlsx

Meetings/

Weekly Meeting.pdf

Company Brain starts reading everything.

Now let's see what happens.

STEP 1 — PostgreSQL Database (16 Tables)

Many beginners think:

"Why can't I store everything in one table?"

Because one table becomes impossible to manage.

Think of a school.

Would you write everything in one notebook?

Student Name

Teacher

Marks

Attendance

Subjects
Fees
Parents
Library Books

No.

School has separate registers.

Similarly Company Brain has separate tables.

Table 1 — graph_entities

This stores ALL important things inside the company.

Example

Alice

Project Phoenix

Microsoft

Legal Approval

Finance Team

Customer ABC

Laptop

Contract #102

Everything important is an Entity.

Example table

Entity ID	Name	Type
1	Alice	Person
2	Project Phoenix	Project
3	Microsoft	Company
4	Legal Approval	Task

Notice:

No relationships.

Only important objects.

Table 2 — graph_entity_aliases

Sometimes the same company has multiple names.

Example

Document 1

Microsoft Corporation

Document 2

Microsoft

Document 3

MSFT

If you store these separately,

Company Brain thinks

Company A

Company B

Company C

Wrong.

So Alias Table stores

Alias	Canonical Entity
MSFT	Microsoft
Microsoft Corporation	Microsoft

Now everything becomes

Microsoft

One company.

Table 3 — graph_relationships

Now connect entities.

Example

Alice

↓

works_on

↓

Project Phoenix

Another

Project Phoenix

↓

depends_on

↓

Legal Approval

Table

Source	Relationship	Target
Alice	works_on	Project Phoenix
Project Phoenix	depends_on	Legal Approval

Now Company Brain understands connections.

Table 4 — graph_relationship_sources

Question:

Where did this relationship come from?

Maybe

Meeting Notes

Maybe Slack

Maybe Email

Maybe Jira

Need proof.

Example

```
Alice
↓
works_on
↓
Project Phoenix
```

Source

```
Meeting_15_July.pdf
```

Table

Relationship	Document
Alice works_on Phoenix	Meeting.pdf

Now AI can explain

"I found this relationship in Meeting.pdf"

Table 5 — graph_facts

Relationships are NOT facts.

Fact means

Revenue

=

\$2.8M

or

Budget

=

\$500,000

or

Project Status

=

Active

Table

Entity	Fact	Value
Project Phoenix	Budget	\$2.8M
Project Phoenix	Status	Active

Table 6 — graph_fact_sources

Again

Where did Budget come from?

Maybe

Budget.xlsx

Store source.

Now Company Brain always knows

which document created which fact.

Table 7 — graph_conflicts

Suppose

Document A

Budget

\$2.8M

Document B

Budget

\$3.1M

Should AI delete one?

NO.

Instead

Store

Conflict

Evidence A

Budget.xlsx

Evidence B

Meeting.pdf

Later a human decides.

Nothing is lost.

Table 8 — graph_decisions

Companies make decisions every day.

Example

Meeting

↓

Decision

↓

Move to AWS

↓

Reason

↓

Better Scalability

Months later CEO asks

"Why did we move to AWS?"

Company Brain answers immediately.

Table 9 — graph_action_recommendations

Company Brain doesn't only understand.

It suggests.

Example

Contract expires

↓

Recommendation

Renew Contract

Another

Project has no owner

↓

Assign Owner

Table 10 — graph_nodes

Imagine graph like

Alice

Project

Finance

These circles are Nodes.

Table 11 — graph_edges

Lines connecting nodes.

Alice

——works_on——>

Project Phoenix

Edges store those lines.

Table 12 — graph_workflows

Suppose company has onboarding.

HR

↓

Manager Approval

↓

Laptop

↓

Email

↓

Employee Joined

This becomes one workflow.

Company Brain learns

How company works.

Table 13 — graph_knowledge_gaps

Suppose

Project exists

No Owner

Company Brain stores

Knowledge Gap

Project Phoenix

Owner Missing

Another

Customer

No Contract

Gap detected.

Table 14 — graph_learning_signals

Suppose AI says

Owner

Alice

Human corrects

No

Owner is Bob

Store feedback.

Future extraction becomes smarter.

Table 15 — graph_versions

Company changes.

Old Policy

Work From Home

3 Days

New Policy

5 Days

Keep both.

Version history.

Never lose information.

Table 16 — graph_domain_ontologies

This is like a dictionary.

Suppose

Engineering

has

Repository

Sprint

Bug

Release

HR has

Employee

Manager

Leave

Salary

Finance has

Budget

Revenue

Profit

Ontology tells Company Brain

which entities belong to each business domain.

Now Let's Understand the Engines

Incremental Knowledge Sync Engine

This is probably the MOST important engine.

Suppose Google Drive initially has

Project.docx

Tomorrow

Project.docx

Budget changed

Should Company Brain rebuild everything?

No.

Connector says

FILE_UPDATED

Knowledge Sync Engine checks

Old File Hash

↓

ABC123

New File Hash

ABC999

Different.

Only that file goes through Module 1 → Module 2 → Module 3.

Everything else stays untouched.

This saves enormous computation.

SHA-256

Think of SHA-256 as a **digital fingerprint** of a file.

Imagine two files:

Budget.xlsx

Yesterday:

Budget = \$2.8M

SHA-256:

```
7FA921...
```

Today, someone changes the budget:

```
Budget = $3.5M
```

New SHA-256:

```
A18C5E...
```

The fingerprints are different, so Company Brain knows the content changed and needs reprocessing.

If only the file permissions changed and the content stayed the same, the SHA-256 fingerprint would also stay the same, so the system skips unnecessary work.

Hybrid Extractor

Document

```
Alice approved Project Phoenix.
```

```
Budget = $2.8M.
```

Rule

```
Budget
```

```
$
```

```
Number
```

Regex extracts Budget immediately.

Meeting Plugin extracts

```
Decision
```

```
Owner
```

```
Action Items
```

Financial Plugin extracts

Revenue

Expenses

Profit

Different plugins understand different document types.

Entity Resolution

Documents say

Microsoft

MSFT

Microsoft Corporation

System merges



Microsoft

One company.

Conflict Detector

Two documents disagree.

Revenue

\$2.8M

Another

Revenue

\$3.1M

Store BOTH.

Human decides later.

Knowledge Reasoner

Suppose

Project Phoenix

↓

depends_on

↓

Legal Approval

Legal Approval becomes overdue.

Nobody wrote

Project Phoenix is at Risk

Reasoner thinks

Legal delayed

↓

Project delayed

↓

Risk

Creates new knowledge automatically.

Impact Analyzer

Suppose

Policy changes.

Instead of manually checking everything,

AI follows relationships.

Policy

↓

Engineering

↓

VPN

↓

Security

↓

IT

Now management knows exactly what is affected.

Organizational Health

Looks at the entire company and calculates things like:

- How complete is our documentation?
- How much knowledge is outdated?
- How many unresolved conflicts exist?
- Are there projects without owners?
- Are there missing workflows?

It then computes an overall **Knowledge Quality Score** (for example, 86/100) to summarize the health of the organization's knowledge.

Decision Intelligence

Instead of remembering only **what** happened, it stores **why** it happened.

For every important decision, it can record:

- The decision itself
- Who made it
- The rationale
- The supporting evidence
- The expected outcome

- The eventual outcome (if tracked)

This creates a long-term institutional memory.

Action Recommender

Once problems are identified, the system suggests what should be done next.

Examples:

- Project has no owner → Recommend assigning an owner.
- Contract expires in 10 days → Recommend starting renewal.
- Conflicting policy versions → Recommend human review.

These are recommendations for people or later automation—they are not necessarily executed automatically.

FastAPI APIs

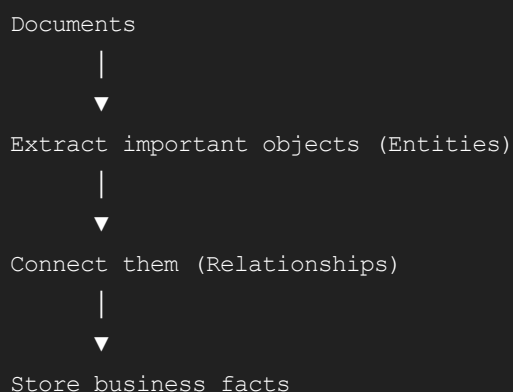
All of this intelligence is exposed through APIs so dashboards, applications, and future AI agents can use it.

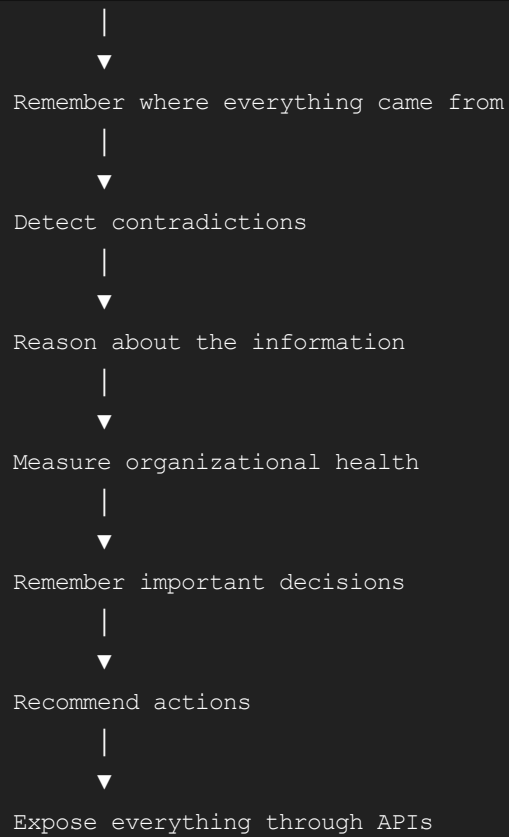
Examples:

- `/api/v1/knowledge/health` → Returns organizational health metrics.
 - `/api/v1/knowledge/impact` → Shows what would be affected by a change.
 - `/api/v1/knowledge/decisions/create` → Records a new institutional decision.
 - `/api/v1/knowledge/recommendations/create` → Stores a recommendation.
-

The Big Picture

Think of Module 3 as building a **digital memory of the company**.





At the end of this process, Company Brain no longer sees "documents." It sees a structured representation of the organization—its people, projects, policies, decisions, dependencies, and knowledge—which is what enables intelligent search, explanations, and future AI agents.